

Methods	Models	Accuracy (%)	Loss
DIVERSE-NLI (Li et al., 2023b)	DeBERTa (He et al., 2020)	61.3	5.43
DIVERSE-NLI (Li et al., 2023b)	LLaMA2-13B	75.6	3.27
DIVERSE-Rule (Li et al., 2023b)	-	75.0	3.43
MATH-SHEPHERD	LLaMA2-13B (N = 4)	85.0	2.05

Table 4: The comparison between NLI/Rule-based automatic process annotation methods from Li et al. (2023b) and our method.

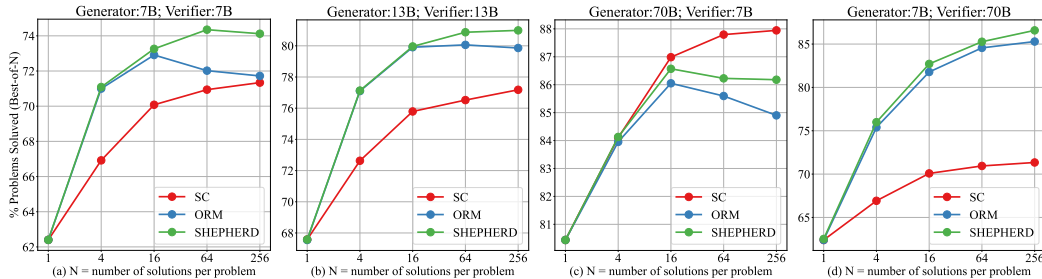


Figure 5: Performance of different verification strategies on different sizes of generators and verifiers.

Figures 5(a), 5(b), and 3(a) display the results from the 7B, 13B, and 70B generators paired with equal-sized reward models, respectively. It becomes evident that PRM exhibits superiority over self-consistency and ORM across all sizes of base models. Moreover, bigger reward models prove to be more robust; for instance, the accuracy of the 70B reward models escalates as the number of candidate solutions rises, while the 7B reward models show a decreasing trend.

Figure 5(c) and 5(d) presents the performance of 7B and 70B generators interfaced with different-sized reward models. The findings illustrate that utilizing a larger reward model to validate the output of a smaller generator significantly enhances performance. Conversely, when a smaller reward model is employed to validate the output of a larger generator, the verification process adversely impacts the model’s performance compared to SC. These results substantiate that we should utilize a more potent reward model for validating or supervising the generator.

5.4 INFLUENCE OF THE NUMBER OF DATA

We delve deeper into the analysis of PRM and ORM by utilizing varying quantities of training data. As depicted in Figure 6(a), it is clear that PRM exhibits superior data efficiency. Specifically, it outperforms ORM by approximately 4% accuracy when applying a modestly sized training dataset (i.e., 10k instances). Furthermore, PRM seems to have a higher potential ceiling than ORM. These observations highlight the efficacy of PRM for verification purposes.

5.5 OUT-OF-DISTRIBUTION PERFORMANCE

To further demonstrate the effectiveness of our method, we conduct an out-of-distribution evaluation on the Hungarian national final exam², which consists of 33 questions. The total score of these questions is 100. We use the LLemma-34B trained on MetaMATH to serve as the generator and generate 256 candidate solutions for each question. We use LLemma-34B-ORM and LLemma-34B-PRM to select the solution for each question. As shown in Figure 6(b): 1) both LLemma-34B-ORM and LLemma-34B-PRM outperform the origin LLemma-34B, showing the reward model can generalize to other domains; 2) PRM outperforms ORM 9 scores, further demonstrating the superiority of PRM.

²https://huggingface.co/datasets/keirp/hungarian_national_hs_finals_exam

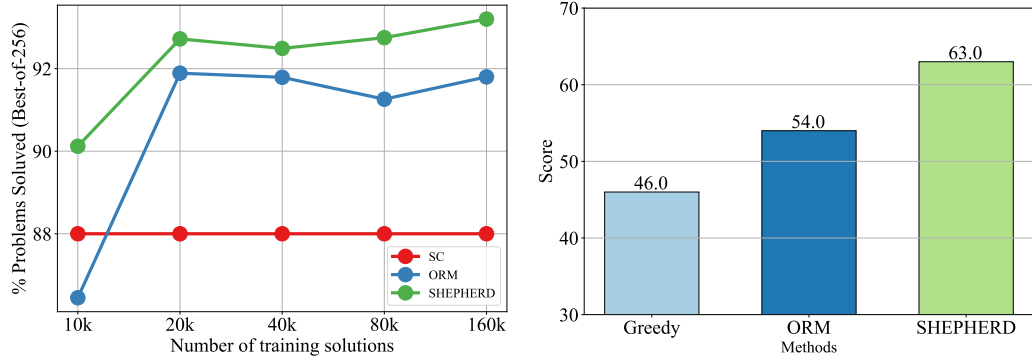


Figure 6: (a): Performance of different reward models using different numbers of training data; (b) performance of different verification strategies on the out-of-distribution Hungarian national exam.

Question	The first term of an arithmetic sequence is 18. The sum of the first six terms of the sequence is equal to the sum of the first seven terms. Show that the sum of the first 13 terms of the sequence is 0 and calculate the 13th term of the sequence.
MATH-SHEPHERD	Step 1: Let the common difference of the arithmetic sequence be d. MATH-SHEPHERD: 0.99. Step 2: The sum of the first six terms of the sequence is $18 + (18 + d) + (18 + 2d) + (18 + 3d) + (18 + 4d) + (18 + 5d) = 108 + 30d$. MATH-SHEPHERD: 0.85. Step 6: Therefore, $d = -3$. MATH-SHEPHERD: 0.85. Step 8: The 13th term of the sequence is $18 - 36 = -18$. MATH-SHEPHERD: 0.93. ORM Score: 0.73.
ORM	Step 1: Let the common difference of the arithmetic sequence be d. MATH-SHEPHERD: 0.99. Step 2: The sum of the first six terms is $18 + (18 + d) + (18 + 2d) + (18 + 3d) + (18 + 4d) + (18 + 5d) = 90 + 15d$. MATH-SHEPHERD: 0.53. Step 6: Dividing by -6, we find that $d = -2$. MATH-SHEPHERD: 0.38. Step 9: The 13th term of the sequence is $18 - 26 = -8$. MATH-SHEPHERD: 0.38. ORM Score: 0.84.

Table 5: A case study from the Hungarian national exam. Red text denotes the mistake that ORM fails to detect.

We also conduct a case study to intuitively demonstrate the effectiveness of MATH-SHEPHERD. As outlined in Table 5, when presented with a question from the Hungarian national final exam, our MATH-SHEPHERD accurately selected the correct solution from a pool of 256 potential solutions, which ORM failed. Moreover, MATH-SHEPHERD displayed superior discernment by precisely identifying incorrect steps within the solutions selected by ORM. Notably, it recognized errors in Step 2, Step 6, and Step 9 and so on, and subsequently assigned them lower scores relative to those for steps present in the correct solutions.

6 LIMITATIONS

Our paper has some limitations, which we leave for future work:

The computational cost of the completion process. To determine the label of each reasoning step, we utilize a ‘completer’ to decode N subsequent reasoning processes. We observe that as N increases, so does the quality of automatic annotations. However, this completion process demands a lot of computing resources, potentially imposing a limitation on the usage of our method. Despite this limitation, the cost remains significantly lower than human annotation. Furthermore, we are optimistic that advancements in efficient inference techniques such as speculative decoding (Xia et al., 2022; Leviathan et al., 2023) and vLLM (Kwon et al., 2023) could mitigate this limitation.

The automatic process annotation consists of noise. Similar to the automatic outcome annotation, our automatic process annotation also has noise. Despite this, our experiments verify the efficacy of our method for training a PRM. In particular, the PRM trained on our dataset outperforms the

human-annotated PRM800K dataset. However, a noticeable gap remains between PRM800K and the candidate responses generated by the open-source models utilized in this study, which may result in the invalidation of PRM800K. As a result, the impact of this potential noise on PRM performance is still undetermined. A comprehensive comparison between human and automated annotations is envisaged for future studies. Furthermore, we assert that integrating human and automated process annotations could play a vital role in constructing robust and efficient process supervision.

7 CONCLUSION

In this paper, we introduce a process-oriented math verifier called MATH-SHEPHERD, which assigns a reward score to each step of the LLM’s outputs on math problems. The training of MATH-SHEPHERD is achieved using automatically constructed process-wise supervision data, thereby eradicating the necessity for labor-intensive human annotation. Remarkably, this automatic methodology correlates strongly with human annotations. Extensive experiments in both verification and reinforcement learning scenarios demonstrate the effectiveness of our method.

REFERENCES

- Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.
- Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *arXiv preprint arXiv:2310.10631*, 2023.
- Zhen Bi, Ningyu Zhang, Yinuo Jiang, Shumin Deng, Guozhou Zheng, and Huajun Chen. When do program-of-thoughts work for reasoning? *arXiv preprint arXiv:2308.15452*, 2023.
- Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.
- Liang Chen, Yichi Zhang, Shuhuai Ren, Haozhe Zhao, Zefan Cai, Yuchi Wang, Peiyi Wang, Tianyu Liu, and Baobao Chang. Towards end-to-end embodied decision making via multi-modal large language model: Explorations with gpt4-vision and beyond. *arXiv preprint arXiv:2310.02071*, 2023.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International conference on computers and games*, pp. 72–83. Springer, 2006.
- DeepSeek. Deepseek llm: Let there be answers. <https://github.com/deepseek-ai/DeepSeek-LLM>, 2023.
- Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. Complexity-based prompting for multi-step reasoning. *arXiv preprint arXiv:2210.00720*, 2022.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Minlie Huang, Nan Duan, Weizhu Chen, et al. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*, 2023.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.